Having built a trie, one can use it in word games.

You can develop a word game such as **Boggle**, where in the set-up phase the computer is asked to generate a NxN (N=4, N=5, N=6) grid of letters with possible repetitions of letters.

In the play of the game, the player has to find words. The rules are:
- Words guessed must be at least three letters in length.
- Each letter chosen in a word after the first letter must be a horizontal, vertical, or diagonal neighbour of the letter before it. No individual instance of a grid letter may be used more than once in a word. That is, if one drew edges from one letter to the next, they must be no revisiting a grid point within one word.
- No capitalised or hyphenated words are allowed.
- Multiple forms of the same word are allowed, such as singular and plural forms and other derivations.

Each word guessed should be checked to see if it is in the dictionary. How will you check that a guessed word is one that is according to the rules (particularly the neighbouring grid point constraint, and no repetitions of a grid point)?

Based on the words guessed that are in the trie, a score can be computed:

Word length	Points
3, 4	1
5	2
6	3
7	5
8	11
9	2 points per letter

Typically the player has to guess a list of words that they can recognise in the grid in a fixed time period, e.g., 3 minutes. Timing may be an add-on feature.
You first need to build a trie dictionary from a given list of words (in a file).

Next you need to generate a NxN grid. When generating the grid, you can use a random letter generator. You may choose to have Q come paired with a u (Qu is what will be in the grid position).

Then, you need a way of recording a player's guesses.

You need to implement a function that will compute a score for a player's guesses. You may also have to decide how to score a word with Qu in it.

Your second assignment is to implement the Boggle game in Rust. The grid setter is the computer, using a random letter generator.

Your tasks are:
(1) to write the main interactive loops, the outer loop for the game comprising the generation of a grid, and displaying it, and then prompting the user for guesses.

(2) Inner interactive loops asking the player for words. It is perhaps worthwhile to not check each word as it is entered, but to collect all the words the player guesses, and then evaluating which are in the dictionary and which are not.
(3) Computing and reporting the score for a player's list of guesses. You may deduct points for guesses that are not words in the dictionary.
(4) making the interactive code **robust**, ensuring that a guess is well formed, and the separators are legitimate (commas, or blank spaces or one word per line). Ill-formed guesses should be rejected politely with appropriate error messages.
(5) documenting the program carefully with in-line comments that provide an abstract description of the code.
(6) including provisions for a good user interface, including one to gracefully exit the game midway, via a special command. One should not allow a guess to be retracted after it has been evaluated.
(7) The minimal interaction medium should be via the terminal ("command-line interaction"). You are welcome to try fancier user interfaces at your leisure.